

Análisis de la base de datos de Inditex



UNIVERSIDAD COMPLUTENSE MADRID

Laura López Rodríguez y Marina Parra Prieto

ÍNDICE

1. Análisis de requisitos.....	3
2. Diseño conceptual.....	4
3. Diseño lógico.....	8
4. Implementación.....	9

1. Análisis de requisitos

Gestión de la empresa Inditex

La empresa Inditex desea desarrollar una base de datos que permita gestionar sus principales operaciones. Este trabajo consiste en controlar la información sobre las tiendas, empleados, clientes, productos, proveedores y ventas, con el fin de optimizar los procesos logísticos y mejorar la eficiencia en la gestión de las tiendas.

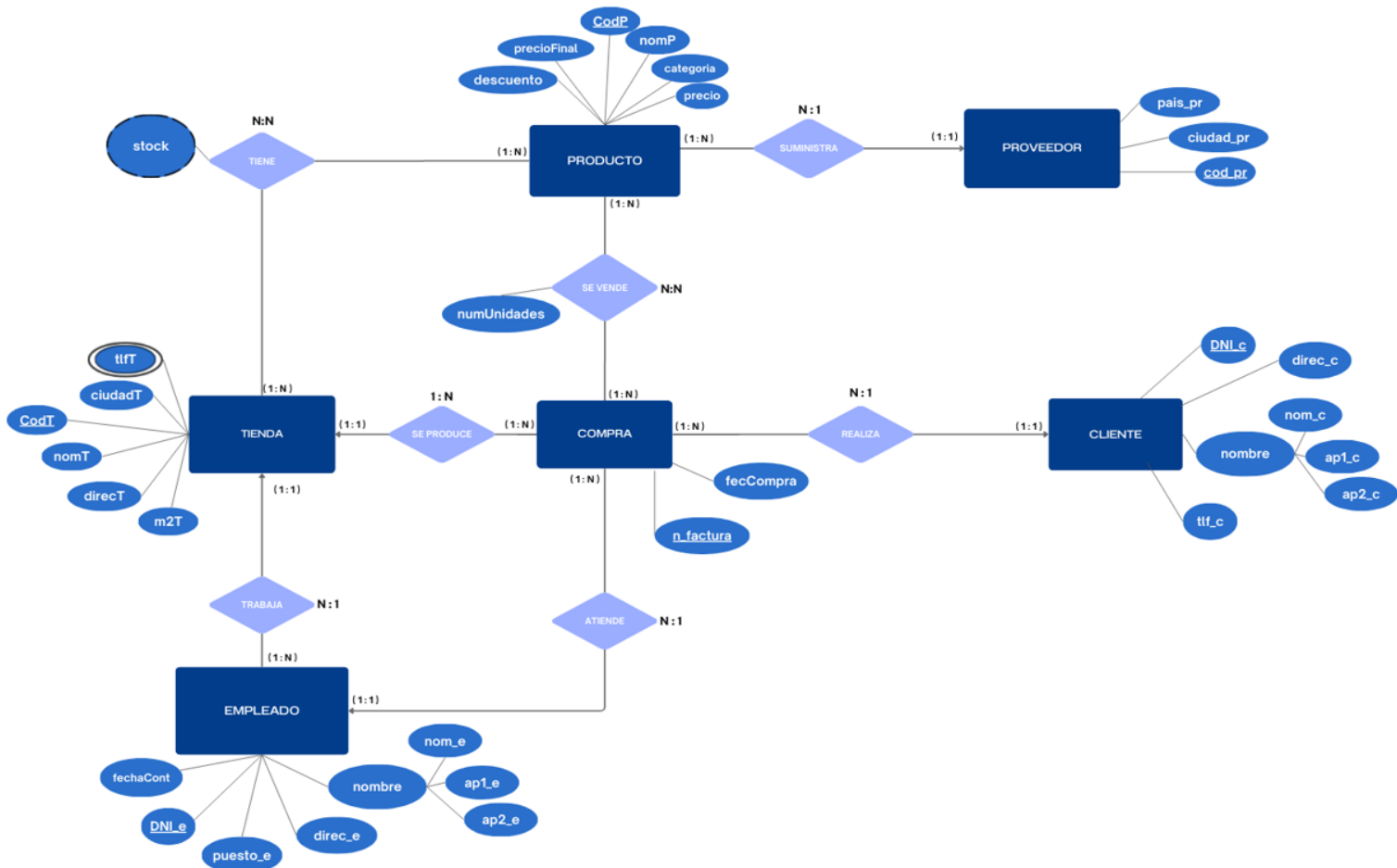
Por lo tanto, las entidades a analizar serán las siguientes:

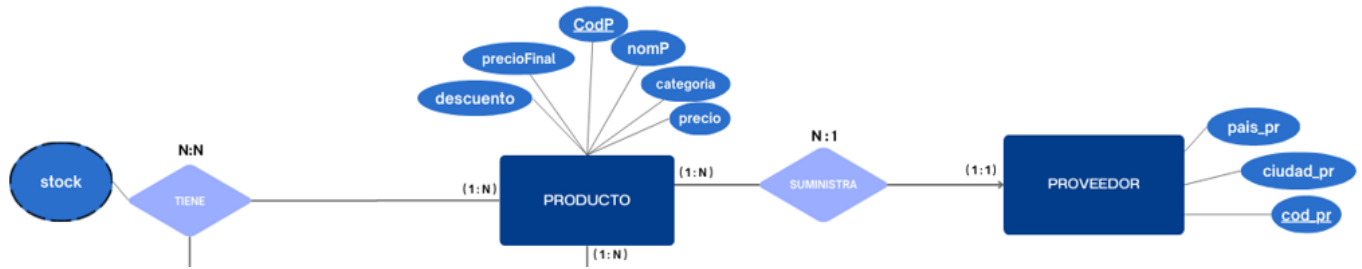
- **Tiendas:** Cada tienda está identificada por un código único y se almacenan datos como su nombre, dirección, teléfonos de contacto, ciudad y área en metros cuadrados. Las tiendas son el punto de venta principal, que cuentan con empleados que realizan las operaciones diarias. Dichas tiendas disponen de varios productos.
- **Empleados:** Cada empleado está identificado por su DNI y trabaja exclusivamente en una tienda. Se almacenan datos como su nombre, apellidos, dirección, fecha de contratación y puesto. Estos llevan a cabo las ventas de los productos de la tienda.
- **Clientes:** La información de interés de los clientes es: su nombre y apellidos, DNI, dirección y teléfono. Los clientes pueden realizar varias compras en diferentes tiendas, y cada vez que hacen una compra se guarda información sobre esta (factura, productos comprados...).
- **Productos:** Los productos están definidos por un código único, nombre, categoría (ropa, calzado, accesorios, etc.) y precio. Cada producto está disponible en diferentes cantidades en las tiendas con su respectivo stock, además, su precio puede verse afectado cuando hay ofertas. Un producto se puede vender en varias compras y una compra puede ser de uno o varios productos.
- **Compras:** Se sabe que una compra es registrada con un número de factura único, además de conocerse la fecha en la que se realizó la compra, el número de unidades que se compraron y la ubicación de esta. Una compra es realizada por un único cliente, pero un cliente puede realizar varias compras.
- **Proveedor:** Los proveedores pueden ser diferentes, y cada proveedor puede suministrar a diferentes almacenes. Están definidos por un código único y conocemos datos como su país y su ciudad de origen.

2. Diseño conceptual

A partir del enunciado anterior hemos realizado el siguiente modelo Entidad – Relación, que ha sido creado con Canva.

Diagrama de entidad-relación





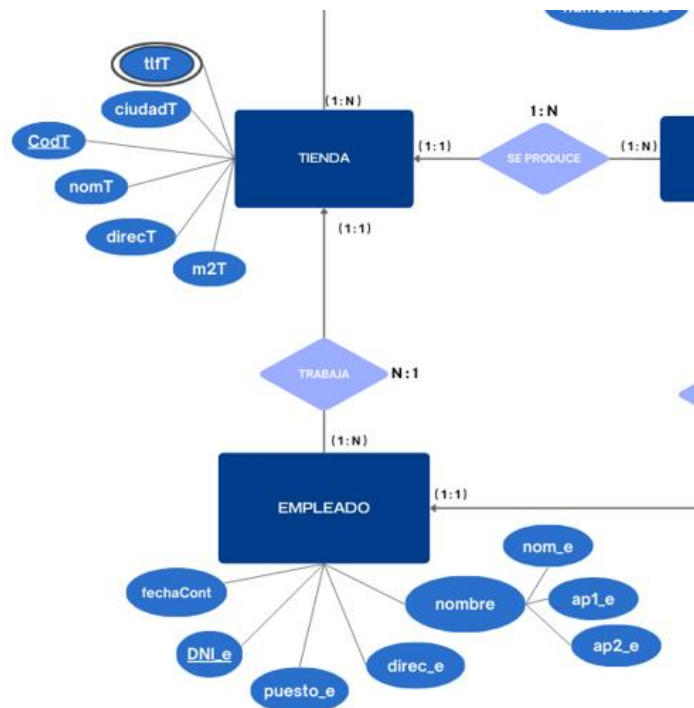
Proveedores:

La información que se recogerá de los proveedores será tanto su país como su ciudad de origen, y el código del proveedor, que será la clave primaria. Encontramos a su vez una relación N:N “Suministra” entre las entidades “Proveedor” y “Producto”, ya que un proveedor puede suministrar 1 producto como mínimo y N como máximo, y un producto es suministrado únicamente por un proveedor.

Producto:

Los productos son suministrados por los proveedores a las tiendas, los identificamos por el código único del producto (CodP). Además, sabremos el nombre del producto (nomP), la categoría y el precio. Por si se lleva a cabo alguna promoción, hemos añadido los atributos descuento y precioFinal, descuento por defecto será 0 a la hora de crear las tablas.

Como un producto puede estar en una tienda como mínimo, y en N tiendas como máximo, y a su vez una tienda puede tener de uno a N productos, surge una relación “Tiene” N:N entre estos dos conceptos. El stock es un atributo derivado, ya que va cambiando según se hacen compras o aumenta el inventario.



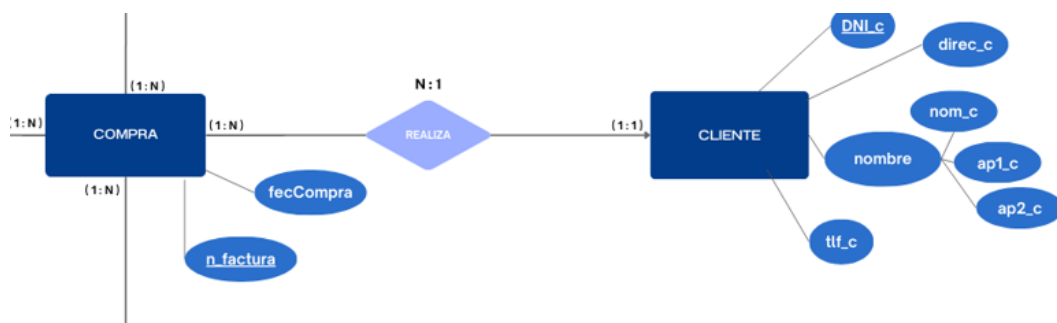
Tienda:

La tienda tiene como clave primaria el código de la tienda (CodT), conocemos también su nombre (nomT) y sus teléfonos (tlfT), la ciudad y la dirección (direcT) en la que está ubicada, así como los metros cuadrados que ocupa la instalación.

En la tienda trabajan varios empleados, y cada empleado trabaja en una única tienda, por tanto nos encontramos ante una relación N:1, que hemos denominado “Trabaja”.

Empleado:

Identificamos a cada empleado con su DNI (DNI_e), y sabemos su puesto, dirección (direc_e), nombre y apellidos, y su fecha de contratación (fechaCont). Surge entonces, la relación “Atiende” N:1 entre “Empleado” y “Compra” ya que los empleados trabajan en una única tienda, y la tienda tiene varios trabajadores que pueden atender una compra.



Cliente:

Identificamos a los clientes por su DNI (DNI_c), y recogemos la siguiente información de estos: dirección (direc_c), nombre y apellidos y su teléfono (tlf_c). Los clientes realizan

las compras en la tienda, un cliente puede hacer el número de compras que desee, pero la compra es realizada por un único cliente. Por tanto, encontramos una relación N:1 "Realiza" entre las entidades "Compra" y "Cliente"

Compra:

Las compras realizadas por los clientes se identifican mediante un número de factura. Conocemos también la fecha en la que se realizó la compra (fecCompra) y el número de unidades que se compraron.

Como hemos mencionado anteriormente, la compra se lleva a cabo en una sola tienda, y se pueden realizar como mínimo una compra en esa tienda y como máximo N. De esta manera nos encontramos ante una relación 1:N "SeProduce" entre las entidades "Tienda" y "Compra".

3. Diseño lógico.

Ahora vamos a analizar todas las entidades y relaciones que componen nuestra base de datos:

Modelo Relacional: “Paso a tablas”

1. Entidades

Proveedor(cod_pr, ciudad_pr, pais_pr)

Tienda(CodT, nomT, direcT, ciudadT, m2T)

TelefonoTienda(tlfT, CodT)

Producto(CodP, nomP, categoria, precio, descuento, precioFinal, Cod_pr)

Compra(n_factura, fecCompra, DNI_e, DNI_c, CodT)

Cliente(DNI_c, direc_c, tlf_c, nom_c, ap1_c, ap2_c)

Empleado(DNI_e, nomb_e, ap1_e, ap2_e, direc_e, puesto_e, fechaCont, CodT)

El paso a tabla de las entidades es el siguiente. En la entidad proveedor tenemos la clave primaria cod_pr y los atributos ciudad_pr, pais_pr. En Tienda tenemos como clave primaria el código de la tienda (CodT) y como atributos el nombre de la tienda, la dirección, la ciudad y los metros cuadrados.

Además, como los teléfonos de la tienda es un atributo multivalorado y una tienda puede tener varios números de teléfono, genera la tabla TelefonoTienda donde ambas son claves primarias: el teléfono de la tienda (tlfT) y el código de la tienda (CodT).

La entidad Producto genera tabla, con el código del producto (CodP) como clave primaria y sus atributos nomP, categoria, precio, descuento, precioFinal. Por otro lado, la clave primaria de la tabla compra es n_factura y el atributo fecha de Compra (fecCompra). Encontramos tres claves ajenas en Compra, que provienen de las entidades Empleado, Cliente y Tienda, las marcamos en azul y subrayadas.

La tabla cliente tiene como clave primaria el DNI del cliente (DNI_c) y como atributos su dirección, su número de teléfono, su nombre y apellidos.

Por último, tenemos la tabla empleado, con su respectiva clave primaria (DNI_e) y la clave ajena CodT proveniente de la tabla tienda por la relación 1:N entre tienda y empleado.

2. Relaciones

Tiene([CodP](#), [CodT](#), stock)

SeVende([n_factura](#), [CodP](#), numUnidades)

Las relaciones N:N generan tabla, en nuestro caso “Tiene” es una relación entre las entidades Producto y Proveedor. Una tienda puede tener un producto como mínimo y N productos como máximo, y un producto puede estar en una tienda como mínimo y en N tiendas como máximo. De esta manera se genera la relación N:N, que tiene como atributo derivado el stock, que modificaremos más tarde con su trigger correspondiente.

Por otro lado, SeVende es una relación N:N entre las entidades Producto y Compra. Observamos además que cuenta con un atributo (numUnidades) que nos da información sobre el número de unidades de producto en la compra.

4. Implementación

Vamos a realizar la implementación en el programa MySQL,

```
/* -----
```

Base de datos para la gestión de INDITEX

Autores: Laura López y Marina Parra

```
-----*/
```

```
/* ----- Definición de la estructura de la base de datos -----
```

```
-----*/
```

```
drop database if exists inditex;
```

```
create database inditex;
```

```
use inditex;
```

```
-- creamos la tabla tienda
```

```
drop table if exists tienda;
```

```
create table tienda (
```

```
CodT int primary key,
```

```
nomT varchar(50),
```

```
direcT varchar(50),
```

```
ciudadT varchar(30),
```

```
m2T integer not null
```

```
);
```

```
-- creamos la tabla TelefonoTienda ya que al ser un atributo multivalorado, genera tabla
```

```

drop table if exists TelefonoTienda;
create table TelefonoTienda (
    tlfT varchar(12),
    codT int,
    foreign key (codT) references tienda(codT)
        on delete cascade
        on update cascade,
    primary key(tlfT, codT)
);

-- creamos la tabla cliente
drop table if exists cliente;
create table cliente (
    DNI_c varchar(9) primary key,
    nom_c varchar(15),
    ap1_c varchar(15),
    ap2_c varchar(15),
    telf_c numeric(9,0),
    direc_c varchar(50)
);

#creamos la tabla empleado
drop table if exists empleado;
create table empleado (
    DNI_e varchar(9) primary key,
    idTienda int,
        foreign key (idTienda) references tienda(CodT),
    nom_e varchar(15),
    ap1_e varchar(15),
    ap2_e varchar(15),
    puesto_e text(20),
    direc_e varchar(50),
    fechaCont date not null
);

#creamos la tabla Proveedor
drop table if exists Proveedor;
create table Proveedor (
    cod_pr int primary key,
    ciudad_pr varchar(30) not null,
    pais_pr varchar (30) not null

```

```
);
```

```
-- creamos la tabla producto
```

```
drop table if exists Producto;
```

```
create table Producto (
```

```
    codP varchar(4) primary key,
```

```
    cod_pr int,
```

```
        foreign key (cod_pr) references proveedor(cod_pr),
```

```
    nomP varchar(40) not null,
```

```
    categoria varchar(30) not null,
```

```
    precio numeric(6,2) not null,
```

```
    descuento numeric(5,2) DEFAULT 0,
```

```
    precioFinal numeric(7,2) DEFAULT (PRECIO-PRECIO*(DESCUENTO/100))
```

```
);
```

```
-- creamos la tabla para la entidad compra
```

```
drop table if exists compra;
```

```
create table compra (
```

```
    n_factura varchar(30) primary key,
```

```
    fecCompra date,
```

```
    id_cliente varchar(30),
```

```
        foreign key (id_cliente) references cliente(DNI_c) on delete cascade,
```

```
    id_producto varchar(4),
```

```
        foreign key (id_producto) references Producto(codP) on delete cascade,
```

```
    id_tienda int,
```

```
        foreign key (id_tienda) references tienda(CodT),
```

```
    id_empleado varchar(9),
```

```
        foreign key (id_empleado) references empleado(DNI_e)
```

```
);
```

```
-- Relación "tiene" entre Tienda y Producto
```

```
drop table if exists tiene;
```

```
create table tiene(
```

```
    CodT int,
```

```
        foreign key (CodT) references tienda(CodT) on delete cascade on update
```

```
cascade,
```

```
    codP varchar(4),
```

```
        foreign key (codP) references producto(codP) on delete cascade on update
```

```
cascade,
```

```
    stock int,
```

```

primary key(CodT, codP)
);

-- Relación "seVende" entre Compra y Producto
drop table if exists seVende;
CREATE TABLE seVende (
    n_factura varchar(30),
    codP varchar(4),
    numUnidades int default 1,
    primary key(n_factura, codP),
    foreign key (n_factura) references compra(n_factura) on delete cascade on update
    cascade,
    foreign key (codP) references Producto(codP) on delete cascade on update cascade
);

```

```

DELIMITER //

```

```

CREATE TRIGGER actualizar_stock
AFTER INSERT ON seVende
FOR EACH ROW
BEGIN
    -- Actualizamos el stock en la tabla TIENE cada vez que se realice una compra
    UPDATE TIENE
    SET stock = stock - NEW.numUnidades
    WHERE codP = NEW.codP;

    -- Lanzaremos un error si el stock es insuficiente
    IF (SELECT stock FROM TIENE WHERE codP = NEW.codP) < 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Error: El stock es insuficiente.';
    END IF;
END//

```

```

DELIMITER ;

```

```

#VAMOS A CREAR UN TRIGGER PARA QUE COMPRUEBE QUE ANTES DE INSERTAR EN
LA TABLA COMPRA, VEA QUE LA TIENDA DEL EMPLEADO
-- EN LA QUE ESTÁ ATENDIENDO LA COMPRA, COINCIDA CON LA TIENDA DE LA
COMPRA EN LA QUE SE ESTÁ REALIZANDO LA COMPRA.
DELIMITER //

```

```

CREATE TRIGGER verificar_tienda_empleado
BEFORE INSERT ON compra
FOR EACH ROW
BEGIN
    DECLARE mismaClave INT;

    -- Obtenemos la tienda asociada al empleado que está atendiendo
    SELECT idTienda INTO mismaClave
    FROM empleado
    WHERE DNI_e = NEW.id_empleado;

    -- Comprobamos que la tienda del empleado coincide con la tienda de la compra
    IF mismaClave IS NULL THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Error: El empleado no existe o no está asociado a ninguna
tienda.';
    ELSEIF mismaClave <> NEW.id_tienda THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Error: La tienda del empleado no coincide con la tienda de
la compra.';
    END IF;
END;
//

```

```

DELIMITER ;

```

```

-- tienda
INSERT INTO tienda VALUES ('16920', 'ZARA', 'C.Conde de Peñalver, 16', 'Madrid', 800);
INSERT INTO tienda VALUES ('59423', 'PULL&BEAR', 'C.Gran Via, 34', 'Madrid', 1383);
INSERT INTO tienda VALUES ('88913', 'BERSHKA', 'C.Serrano, 23', 'Madrid', 1000);
INSERT INTO tienda VALUES ('23547', 'STRADIVARIUS', 'C.C La Vaguada', 'Madrid', 900);
INSERT INTO tienda VALUES ('56971', 'STRADIVARIUS', 'C.Gran Via, 25', 'Madrid', 856);
INSERT INTO tienda VALUES ('13679', 'BERSHKA', 'C.Gran Via, 32', 'Madrid', 989);
INSERT INTO tienda VALUES ('19736', 'ZARA', 'C.Preciados, 13', 'Madrid', 1006);
INSERT INTO tienda VALUES ('48261', 'ZARA', 'C.Orense, 27', 'Madrid', 500);
INSERT INTO tienda VALUES ('97138', 'STRADIVARIUS', 'C.Alcalá, 161', 'Madrid', 200);
INSERT INTO tienda VALUES ('29944', 'OYSHO', 'C.Antonio López, 16', 'Madrid', 400);

```

```

-- TelefonoTienda

```

```

INSERT INTO TelefonoTienda VALUES (937846308, '16920');
INSERT INTO TelefonoTienda VALUES (914565308, '59423');
INSERT INTO TelefonoTienda VALUES (917845208, '88913');
INSERT INTO TelefonoTienda VALUES (914849303, '23547');
INSERT INTO TelefonoTienda VALUES (914394943, '56971');
INSERT INTO TelefonoTienda VALUES (913389307, '13679');
INSERT INTO TelefonoTienda VALUES (917845308, '19736');
INSERT INTO TelefonoTienda VALUES (915632478, '48261');
INSERT INTO TelefonoTienda VALUES (913246755, '97138');
INSERT INTO TelefonoTienda VALUES (952133477, '29944');

```

-- CLIENTE

```

INSERT INTO cliente VALUES ('05978507A', 'Eva', 'Moruga', 'Rodríguez', 632148972,
'Rávena 10');
INSERT INTO cliente VALUES ('87208136C', 'Carolina', 'Rodríguez', 'Martinez',
624565334, 'Lucano 14');
INSERT INTO cliente VALUES ('88362017G', 'Eugenia', 'Parra', 'Prieto', 667845212,
'Esteban Collantes 35');
INSERT INTO cliente VALUES ('61882330L', 'Angela', 'Martínez', 'Yebra', 665713248,
'Quinto 8');
INSERT INTO cliente VALUES ('03674561H', 'Veronica', 'Madrigal', 'Carro', 914394943,
'Julián Camarillo 11');
INSERT INTO cliente VALUES ('17844509J', 'Ignacio', 'Martín', 'Zarzalejos', 605656441,
'Aquitania 21');
INSERT INTO cliente VALUES ('18990367D', 'Daniela', 'Garcia', 'López', 623389319,
'Alcalá 57');
INSERT INTO cliente VALUES ('68229461K', 'Juan', 'Martín', 'Arranz', 640845323, 'Arturo
Soria 50');
INSERT INTO cliente VALUES ('61036771N', 'Marcos', 'Gonzalez', 'Castillo', 697412399,
'Nápoles 3');
INSERT INTO cliente VALUES ('36189026Q', 'Aitana', 'Diaz', 'Gonzalez', 617823399,
'Aracne 16');
INSERT INTO cliente VALUES ('56183029A', 'Eduardo', 'Calviño', 'Rodríguez', 638923420,
'Suecia 20');
INSERT INTO cliente VALUES ('05977509Q', 'Juana', 'Prieto', 'Álvarez', 638890420,
'Olmos 20');

```

-- empleado

```

INSERT INTO empleado VALUES ('65478912H', '16920', 'Ainara', 'Barreche', 'Rodríguez',
'Cajera', 'C.San Germán, 10', '2017-03-15');

```

```

INSERT INTO empleado VALUES ('02645971K', '59423', 'Alejandra', 'Serrano', 'González',
'Dependienta', 'C.Navalmoral de la Mata, 16', '2018-07-20');
INSERT INTO empleado VALUES ('59741623A', '88913', 'Julio', 'Gomez', 'Muñoz',
'Responsable de sección', 'C.de Ávila, 8', '2019-11-05');
INSERT INTO empleado VALUES ('89512364F', '23547', 'Angela', 'González', 'Ortega',
'Dependienta', 'C.del Plátano, 20', '2020-02-28');
INSERT INTO empleado VALUES ('26459781Z', '56971', 'Raúl', 'Sanz', 'Fernández',
'Cajero', 'C.de Finisterre, 11', '2017-06-10');
INSERT INTO empleado VALUES ('36985214V', '13679', 'Jorge', 'Martín', 'Mendoza',
'Encargado', 'C.Suiza, 19', '2021-09-15');
INSERT INTO empleado VALUES ('14785239D', '19736', 'Iván', 'Gonzalez', 'Gómez',
'Mozo', 'C.Estocolmo, 37', '2019-12-22');
INSERT INTO empleado VALUES ('78123694R', '48261', 'María', 'Hernández', 'Ruiz',
'Dependienta', 'C. de Jaén, 60', '2022-01-12');
INSERT INTO empleado VALUES ('79134562T', '97138', 'Eric', 'Plaza', 'Castillo',
'Dependiente', 'C. del Panizo, 3', '2020-08-03');
INSERT INTO empleado VALUES ('13791026S', '29944', 'Pablo', 'Diaz', 'García', 'Cajero',
'C.de Sarria, 30', '2018-04-18');

```

-- proveedor

```

INSERT INTO Proveedor(cod_pr, ciudad_pr , pais_pr) VALUES ('47079', 'Ankara',
'Turquía');
INSERT INTO Proveedor(cod_pr, ciudad_pr , pais_pr) VALUES ('25749', 'Madrid',
'España');
INSERT INTO Proveedor(cod_pr, ciudad_pr , pais_pr) VALUES ('55050', 'Nueva Delhi',
'India');
INSERT INTO Proveedor(cod_pr, ciudad_pr , pais_pr) VALUES ('56789', 'Daca',
'Bangladesh');
INSERT INTO Proveedor(cod_pr, ciudad_pr , pais_pr) VALUES ('34021', 'Phnom Penh',
'Camboya');
INSERT INTO Proveedor(cod_pr, ciudad_pr , pais_pr) VALUES ('85231', 'Pekín', 'China');
INSERT INTO Proveedor(cod_pr, ciudad_pr , pais_pr) VALUES ('72028', 'Rabat',
'Marruecos');
INSERT INTO Proveedor(cod_pr, ciudad_pr , pais_pr) VALUES ('11111', 'Lisboa',
'Portugal');

```

-- producto

```

INSERT INTO producto(codP, cod_pr, nomP, categoria, precio) VALUES ('0001', '47079',
'Top Lentejuelas', 'Ropa', 12);
INSERT INTO producto(codP, cod_pr, nomP, categoria, precio) VALUES ('0022', '25749',
'Camisa de satén', 'Ropa', 20);

```

```

INSERT INTO producto(codP, cod_pr, nomP, categoria, precio) VALUES ('0013', '55050',
'Falda Larga', 'Ropa', 25);
INSERT INTO producto(codP, cod_pr, nomP, categoria, precio) VALUES ('0034', '56789',
'Pantalones Wide Leg', 'Ropa', 30);
INSERT INTO producto(codP, cod_pr, nomP, categoria, precio) VALUES ('0105', '85231',
'Gafas de Sol', 'Accesorios', 9);
INSERT INTO producto(codP, cod_pr, nomP, categoria, precio) VALUES ('0096', '72028',
'Pantalón de lino', 'Ropa', 10);
INSERT INTO producto(codP, cod_pr, nomP, categoria, precio) VALUES ('0777', '11111',
'Camisa Crop', 'Ropa', 30);
INSERT INTO producto(codP, cod_pr, nomP, categoria, precio) VALUES ('0438', '56789',
'Botas Altas', 'Calzado',30);
INSERT INTO producto(codP, cod_pr, nomP, categoria, precio) VALUES ('0161', '25749',
'Cargo Jeans', 'Ropa', 30);
INSERT INTO producto(codP, cod_pr, nomP, categoria, precio) VALUES ('9873', '47079',
'Jersey de punto', 'Ropa', 25);

```

-- compra

```

INSERT INTO compra VALUES ('F0001', '2020-01-15', '05978507A', '0001', 16920,
'65478912H');
INSERT INTO compra VALUES ('F0002', '2019-01-16', '87208136C', '0022', 59423,
'02645971K');
INSERT INTO compra VALUES ('F0003', '2021-01-18', '88362017G', '0013', 88913,
'59741623A');
INSERT INTO compra VALUES ('F0004', '2017-01-20', '61882330L', '0034', 23547,
'89512364F');
INSERT INTO compra VALUES ('F0005', '2015-01-22', '03674561H', '0105', 56971,
'26459781Z');
INSERT INTO compra VALUES ('F0006', '2024-02-01', '17844509J', '0096', 13679,
'36985214V');
INSERT INTO compra VALUES ('F0007', '2020-02-03', '18990367D', '0777', 19736,
'14785239D');
INSERT INTO compra VALUES ('F0008', '2022-02-05', '68229461K', '0438', 48261,
'78123694R');
INSERT INTO compra VALUES ('F0009', '2023-02-10', '61036771N', '0161', 97138,
'79134562T');
INSERT INTO compra VALUES ('F0010', '2023-02-15', '36189026Q', '9873', 29944,
'13791026S');

```

-- relacion seVende

```

INSERT INTO seVende(codP, n_factura, numUnidades) VALUES ('0001', 'F0001', 2);

```

```

INSERT INTO seVende(codP, n_factura, numUnidades) VALUES ('0777', 'F0002', 1);
INSERT INTO seVende(codP, n_factura, numUnidades) VALUES ('0777', 'F0003', 3);
INSERT INTO seVende(codP, n_factura, numUnidades) VALUES ('0438', 'F0004', 1);
INSERT INTO seVende(codP, n_factura, numUnidades) VALUES ('0096', 'F0005', 4);
INSERT INTO seVende(codP, n_factura, numUnidades) VALUES ('0105', 'F0006', 2);
INSERT INTO seVende(codP, n_factura, numUnidades) VALUES ('0022', 'F0007', 1);
INSERT INTO seVende(codP, n_factura, numUnidades) VALUES ('0013', 'F0008', 5);
INSERT INTO seVende(codP, n_factura, numUnidades) VALUES ('0013', 'F0009', 2);
INSERT INTO seVende(codP, n_factura, numUnidades) VALUES ('0034', 'F0010', 3);

```

-- relación tiene

```

INSERT INTO tiene VALUES ('16920', '0001', 100);
INSERT INTO tiene VALUES ('59423', '0022', 90);
INSERT INTO tiene VALUES ('88913', '0013', 500);
INSERT INTO tiene VALUES ('23547', '0034', 500);
INSERT INTO tiene VALUES ('56971', '0105', 100);
INSERT INTO tiene VALUES ('13679', '0096', 350);
INSERT INTO tiene VALUES ('19736', '0777', 275);
INSERT INTO tiene VALUES ('48261', '0438', 475);
INSERT INTO tiene VALUES ('97138', '0161', 400);
INSERT INTO tiene VALUES ('29944', '9873', 300);

```

use inditex;

#1. Muestra los artículos con descuento ordenados por el porcentaje de descuento de forma ascendente

-- 1º Realizamos un descuento en aquellos articulos cuyo precio sea superior a 20 euros

set sql_safe_updates=0;

update producto

set descuento = 50

where precio > 20;

-- 2º calculamos el precio final con el nuevo descuento

update producto

set precioFinal = precio - (precio*descuento / 100);

-- ahora seleccionaos los articulos y su precio final ordenados por el porcentaje de forma ascendente y el nombre en forma descendente

select nomP, precio, descuento, precioFinal

from producto

order by descuento desc, nomP asc;

#2. Muestra el nombre y los apellidos del cliente que ha realizado la compra más reciente

```
select concat(nom_c, " ", ap1_c, " ", ap2_c, "") nombreC, fecCompra
from cliente cl inner join compra c on cl.DNI_c=c.id_cliente
where c.fecCompra=(select fecCompra
                    from compra
                    order by fecCompra desc
                    limit 1);
```

#3. Encuentra los clientes que no han realizado compras

```
select concat(nom_c, " ", ap1_c, " ", ap2_c, "") nombreC
from cliente
where DNI_c not in (select id_cliente
                   from compra);
```

#4. Calcula el precio del producto más caro, el más barato y la media de los dos

```
select max(precio), min(precio), (max(precio) + min(precio))/2 media_precios
from producto;
```

#5. Obtén el telefono de todas las tiendas que se encuentren en Gran Via

```
select tlfT
from telefonotienda tlf inner join tienda t on tlf.codT=t.CodT
where t.direcT like '%Gran Via%';
```

#6. Obtén el nombre de los productos suministrados por el proveedor procedente de Turquía

```
select nomP
from producto p inner join proveedor pr on p.cod_pr = pr.cod_pr
where pais_pr = 'Turquía';
```

#8. Nombre de los artículos de los que se han vendido 3 o más unidades en una compra

```
select nomP
from producto p inner join seVende sV on p.codP=sV.codP
    inner join compra c ON sV.n_factura=c.n_factura
where sv.numUnidades >= 3;
```

#9. Calcula el cliente que más dinero se ha gastado y el nombre de la tienda en la que se ha realizado dicho gasto.

-- Primero calculamos una vista en la que se guarde el gasto pro cliente y en qué tienda se ha realizado

```
create view Gastos_Cliente as
select DNI_c, concat(nom_c, " ", ap1_c, " ", ap2_c, "") nombreC, CodT,
sum(numUnidades*precio) totalGasto
from cliente cl inner join compra c on cl.DNI_c = c.id_cliente
      inner join tienda t on c.id_tienda=t.CodT
      inner join producto p on c.id_producto = p.codP
      inner join seVende sV on p.codP=sV.codP
group by DNI_c, CodT;
```

-- Despues seleccionamos el cliente que más dinero ha gastado y en qué tienda ha sido. Para ello comparamos el gastato total de cada cleinte

```
-- con el máximo del gasto total, y el que coincida será el que más dinero se ha gastado.
select nombreC, nomT, totalGasto
from Gastos_Cliente gpc inner join tienda t on gpc.CodT=t.CodT
where totalGasto = (select max(totalGasto)
                    from Gastos_Cliente);
```

#10. Obtén el el nombre del cliente y el numero de años transcurridos desde la ultima fecha de compra de cada uno de ellos.

```
select nom_c, year(curdate())-year(max(fecCompra)) annosTranscurridos -- le restamos
al año de la fecha actual el año de la ultima fecha de comprpa de cada cliente que es el
max(fecCompra)
from cliente cl inner join compra c on cl.DNI_c = c.id_cliente
group by cl.nom_c;
```

#11. Nombre de los productos que sólo han sido suministrados por el proveedor de Portugal

```
select nomP
from producto p inner join proveedor pr on p.cod_pr=pr.cod_pr
where pais_pr like 'Portugal' and nomP not in(select nomP
                                              from producto p inner join proveedor pr
                                              on p.cod_pr=pr.cod_pr
                                              where pais_pr not like 'Portugal');
```

#12. Obtén los productos que han sido vendidos en al menos una compra, junto con la cantidad total de unidades vendidas de cada producto.

```
select p.codP, p.nomP, sum(numUnidades) totalUdsVendidas
```

```

from producto p inner join seVende sv on p.CodP = sv.CodP inner join compra c
on sv.n_factura = c.n_factura
group by p.codP, p.nomP
order by totalUdsVendidas desc;

```

#13. Calcula el proveedor que ha suministrado el producto con más unidades vendidas en total.

Además, muestra la cantidad total de unidades vendidas de ese producto, el nombre del producto.

-- Primero, creamos una vista para obtenemos el numero de unidades vendidas de cada producto

```

create view unidadesVendidas as
    select p.nomP, p.codP, sum(numUnidades) numVendidas
    from producto p inner join seVende sV on p.codP=sV.codP
    inner join compra c on sV.codP=c.id_producto
    group by codP;

```

-- Ahora seleccionamos el proveedor que ha suministrado el producto mas vendido

```

select cod_pr, numVendidas
from unidadesVendidas uV inner join producto p on uV.codP = p.codP
    inner join compra c on p.codP=c.id_producto
group by cod_pr, numVendidas
having numVendidas = (select numVendidas
                        from unidadesVendidas
                        order by numVendidas desc
                        limit 1);

```

#14. Muestra todos los productos vendidos después del 1 de enero de 2024

```

select nomP , fecCompra, numUnidades
from producto p inner join seVende sv on p.CodP = sv.CodP inner join compra c
on sv.n_factura = c.n_factura
where fecCompra > '2024-01-01' ;

```

#15. Encuentra los clientes que no han realizado compras

```

select DNI_c, concat(nom_c, ' ', ap1_c, ' ', ap2_c, ' ') nombreCliente
from cliente cl left join compra c on cl.DNI_c = c.id_cliente
where id_cliente is null
order by nombreCliente;

```

#16. Modifica el precio de todos los productos comprados en 2020 para aumentar un 10% el precio del producto comprado

```
update producto p inner join compra c on p.codP = c.id_producto
set p.precio = p.precio * 1.1
where year(c.fecCompra) = 2020;
```

#17. Elimina aquellos productos de los que se hayan vendido menos de 3 unidades

```
delete from producto
where codP in (select codP
               from seVende
               group by codP
               having sum(numUnidades) < 3);
```

#18. Lista los datos clientes que se hayan gastado más de 100 euros en inditex

```
select DNI_c, concat(nom_c, ' ', ap1_c, ' ', ap2_c, ' ') nombreC, telf_C telefonoCliente ,
direc_c direcCliente, sum(numUnidades * precio) gastoTotal
from cliente cl inner join compra c on cl.DNI_c = c.id_cliente
               inner join producto p on c.id_producto = p.codP
               inner join seVende sv on p.codP = sv.codP
group by c.id_cliente having sum(numUnidades * precio) > 100
order by gastoTotal desc;
```